Лабораторная работа №1 Моргун Д.Р. ПМ-41
ОС: Windows 11
Docker Client: 29.8.0
Docker Server: 29.8.0 (Engine)

1. Через Docker version смотрим полную и развернутую информацию об установленных компонентах докера (docker --version выводит только версию клиента). Через docker pull скачиваем nginx:alpine из docker hub (nginx - веб-сервис, а alpine - тэг, который означает, что образ собран на базе Alpine Linux) (было в лекции 2 пункт 7, команда nginx:alpine взята из условий лабораторной)

```
PS D:\docker-lab1> docker version

Client:

 Version:           29.8.0

 API version:       1.56

 Go version:        go1.26.8

 Git commit:        88096ef

 Built:             Thu Sep  3 21:53:38 2026

 OS/Arch:           windows/amd64

 Context:           desktop-linux

Server: Docker Desktop 4.91.0 (239619)

 Engine:

  Version:          29.8.0

  API version:      1.56 (minimum version 1.40)

  Go version:       go1.26.8

  Git commit:       3ce5872

  Built:            Thu Sep  3 21:51:20 2026
```

```
 OS/Arch:          linux/amd64
 Experimental:     false
containerd:
 Version:          v2.3.4
 GitCommit:        db8809540e1a7a9da5d518876894933ff55692ab
runc:
 Version:          1.4.3
 GitCommit:        v1.4.3-0-gbb14dabe
docker-init:
 Version:          0.19.0
 GitCommit:        de40ad0
```

```
PS D:\docker-lab1> docker pull nginx:alpine

alpine: Pulling from library/nginx

Digest:
sha256:62ff2089abf5a9ed33bd232895bef5e22f7bb4b200675cec49a5ebc48e3d4ac8

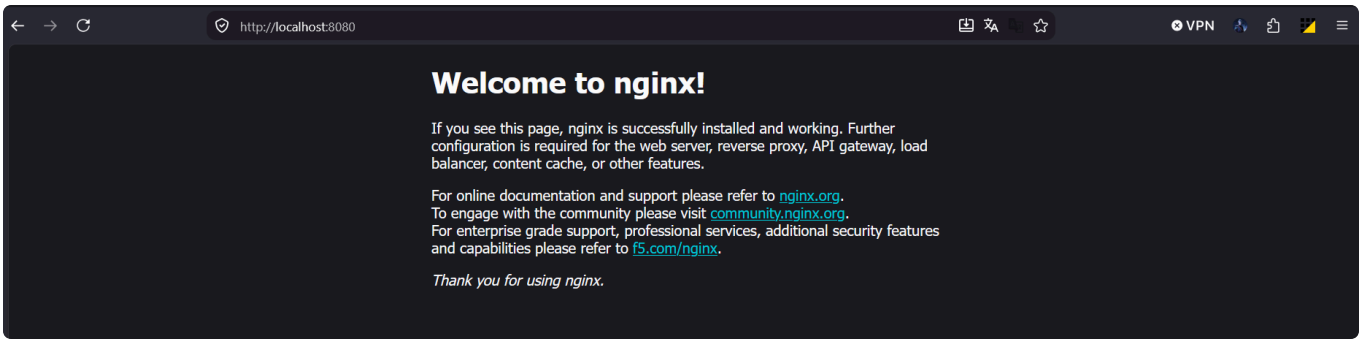
Status: Image is up to date for nginx:alpine

docker.io/library/nginx:alpine
```

2. Запускаем контейнер. Флаг -d обозначает запуск в фоновом режиме, --name присваивает имя, понятное для человека, -p 8080:80 связывает порт 8080 хоста с 80 портом внутри веб-сервера контейнера (лекция 3 пункты 3 и 4). Docker stop останавливает работающий контейнер (в нашем случае по имени). Docker rm удаляем остановленный контейнер (лекция 2 пункты 3 и 4). Docker ps показывает только работающие контейнеры (с флагом -a показывает все, лекция 2 пункт 2)

```
PS D:\docker-lab1> docker run -d --name lab-web-morgun -p 8080:80 nginx:alpine

216170d023296846bb17233f0df5cfd1a149c124c4b271df47145509776d5ca5
```



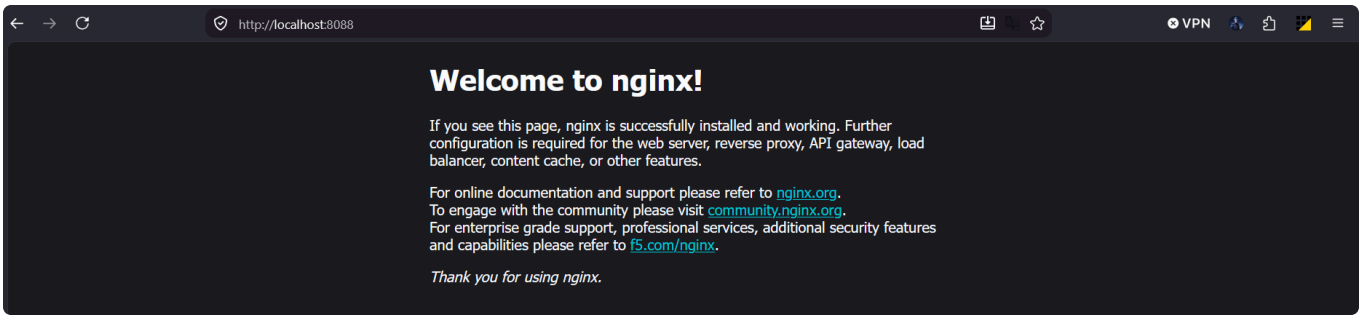

```
PS D:\docker-lab1> docker stop lab-web-morgun

lab-web-morgun
```

```
PS D:\docker-lab1> docker rm lab-web-morgun

PS D:\docker-lab1> docker run -d --name lab-web-morgun -p 8088:80 nginx:alpine

fb8e81187147da88fb623658a80fd1d2568b317fd92ad0f202d3a479c82ef7e3
```




```
PS D:\docker-lab1> docker ps

CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS         PORTS                                     NAMES

fb8e81187147   nginx:alpine   "/docker-entrypoint.…"   2 minutes ago   Up 2
minutes   0.0.0.0:8088->80/tcp, [::]:8088->80/tcp   lab-web-morgun
```

3. Создаем директорию site. Затем через Set-Content записываем Hello there в index.html (лекция 3 пункт 5). Запускаем и через флаг -v "${PWD}\site:/usr/share/nginx/html:ro" связываем папку на компе ({PWD}\site - путь до этой папки) с папкой внутри контейнера (/usr/share/nginx/html - путь внутри контейнера) в режиме "только для чтения" (лекция 3 пункт 5). Если поменять содержимое файла на компе, то оно обновится и в контейнере без всяких перезапусков, так что после обновления страницы будет уже обновленная запись (подтверждение на скриншоте). При

удалении контейнера данные останутся нетронутыми, так как хранятся именно в папке на компе

```
PS D:\docker-lab1> mkdir site

    Каталог: D:\docker-lab1

Mode                 LastWriteTime         Length Name

----                 -------------         ------ ----

d-----        21.09.2026     14:55                site
```

```
PS D:\docker-lab1> "<h1>Hello there</h1>" | Set-Content .\site\index.html
```

```
PS D:\docker-lab1> docker stop lab-web-morgun

lab-web-morgun
```

```
PS D:\docker-lab1> docker rm lab-web-morgun

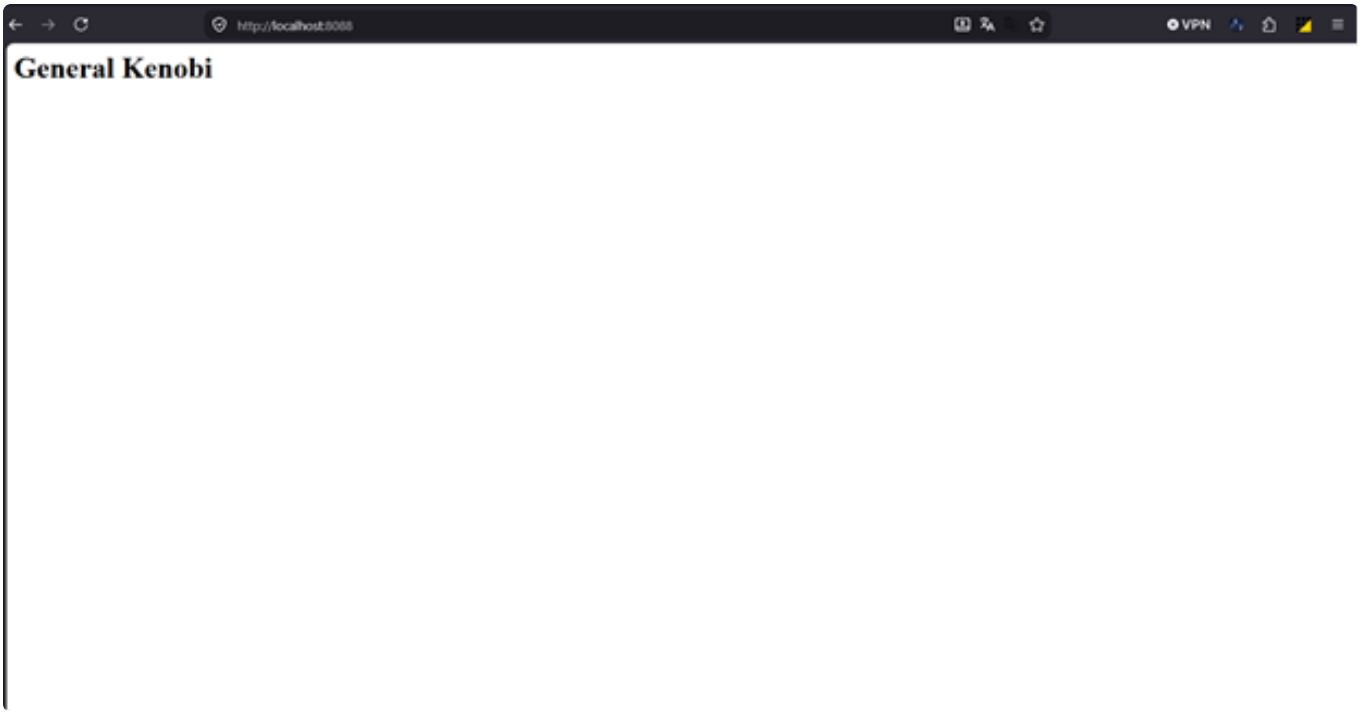
lab-web-morgun
```

```
PS D:\docker-lab1> docker run -d --name lab-web-morgun -p 8088:80 -v
"${PWD}\site:/usr/share/nginx/html:ro" nginx:alpine

032e136d5191c5729ac6cbe0364eb39239a2fc5dd583a46bf447530cf5f42271
```




```
PS D:\docker-lab1> "<h1>General Kenobi</h1>" | Set-Content .\site\index.html
```



4. Через exec запускаем командную оболочку /bin/sh внутри контейнера, чтобы зайти в него и посмотреть содержимое (лекция 2, пункт 9, шаг 3). Флаг -i оставляет ввод открытым и позволяет передавать команды внутрь, а -t подключает псевдотерминал для корректного отображения командной строки и вывода (лекция 3, пункт 2). При вводе touch получаем ошибку Read-only file system, так как контейнер запущен в режиме только для чтения. При перезапуске в режиме чтения и записи файл спокойно создается (подтверждено командой Get-ChildItem)

```
PS D:\docker-lab1> docker exec -it lab-web-morgun /bin/sh

/ # ls -la /usr/share/nginx/html

total 4

drwxrwxrwx    1 root     root           512 Sep 21 11:55 .

drwxr-xr-x    3 root     root          4096 Sep 17 21:16 ..

-rwxrwxrwx    1 root     root            25 Sep 21 12:01 index.html

/ # touch /usr/share/nginx/html/test.txt

touch: /usr/share/nginx/html/test.txt: Read-only file system

/ # exit

What's next:

    Try Docker Debug for seamless, persistent debugging tools in any container
or image → docker debug lab-web-morgun

    Learn more at https://docs.docker.com/go/debug-cli/
```

```
PS D:\docker-lab1> docker stop lab-web-morgun

lab-web-morgun

PS D:\docker-lab1> docker rm lab-web-morgun

lab-web-morgun
```

```
PS D:\docker-lab1> docker run -d --name lab-web-morgun -p 8088:80 -v
"${PWD}\site:/usr/share/nginx/html" nginx:alpine

49a40c9f612da3589e233a0622231808e7416c86cdf7043c9fa1e318e93ea5cb
```

```
PS D:\docker-lab1> docker exec -it lab-web-morgun /bin/sh

/ # touch /usr/share/nginx/html/test2.txt

/ # exit
```

```
What's next:

    Try Docker Debug for seamless, persistent debugging tools in any container
or image → docker debug lab-web-morgun

    Learn more at https://docs.docker.com/go/debug-cli/
```

```
PS D:\docker-lab1> Get-ChildItem .\site

    Каталог: D:\docker-lab1\site

Mode                 LastWriteTime         Length Name

----                 -------------         ------ ----

-a----        21.09.2026     15:01             25 index.html

-a----        21.09.2026     15:05              0 test2.txt
```

5. Docker logs показывает логи (лекция 3, пункт 8). Флаг --tail 10 показывает только последние 10 записей (там же есть запись про запрос к несуществующей странице). Docker attach прикрепляет терминал к главному процессу контейнера. Чтобы выйти без завершения процесса контейнера есть горячие клавиши Ctrl + P, а затем Ctrl + Q (лекция 2, пункт 10). У меня не получилось их использовать, поэтому я открыл второй терминал и ввел docker ps уже оттуда при активном первом

```
PS D:\docker-lab1> docker logs --tail 10 lab-web-morgun

2026/09/21 12:05:00 [notice] 1#1: start worker process 36

2026/09/21 12:05:00 [notice] 1#1: start worker process 37

172.17.0.1 - - [21/Sep/2026:12:08:19 +0000] "GET / HTTP/1.1" 304 0 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:156.0) Gecko/20100101
Firefox/156.0" "-"

172.17.0.1 - - [21/Sep/2026:12:08:21 +0000] "GET / HTTP/1.1" 304 0 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:156.0) Gecko/20100101
Firefox/156.0" "-"

172.17.0.1 - - [21/Sep/2026:12:08:23 +0000] "GET / HTTP/1.1" 304 0 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:156.0) Gecko/20100101
Firefox/156.0" "-"
```

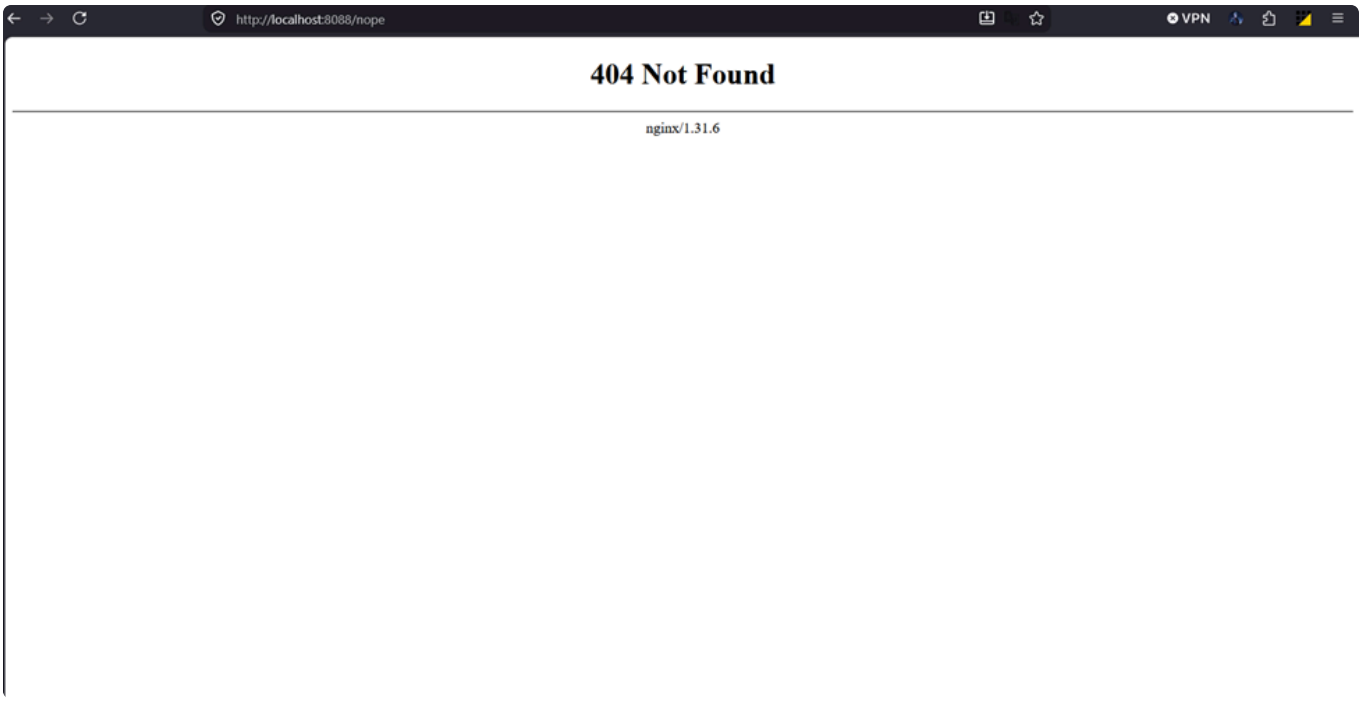
```
172.17.0.1 - - [21/Sep/2026:12:08:25 +0000] "GET / HTTP/1.1" 304 0 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:156.0) Gecko/20100101
Firefox/156.0" "-"

2026/09/21 12:08:33 [error] 30#30: *1 open() "/usr/share/nginx/html/nope"
failed (2: No such file or directory), client: 172.17.0.1, server: localhost,
request: "GET /nope HTTP/1.1", host: "localhost:8088"

172.17.0.1 - - [21/Sep/2026:12:08:33 +0000] "GET /nope HTTP/1.1" 404 153 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:156.0) Gecko/20100101
Firefox/156.0" "-"

172.17.0.1 - - [21/Sep/2026:12:08:35 +0000] "GET /nope HTTP/1.1" 404 153 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:156.0) Gecko/20100101
Firefox/156.0" "-"

2026/09/21 12:08:35 [error] 30#30: *1 open() "/usr/share/nginx/html/nope"
failed (2: No such file or directory), client: 172.17.0.1, server: localhost,
request: "GET /nope HTTP/1.1", host: "localhost:8088"
```




```
PS C:\Users\dimam> docker attach lab-web-morgun

172.17.0.1 - - [21/Sep/2026:12:15:01 +0000] "GET / HTTP/1.1" 304 0 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:156.0) Gecko/20100101
Firefox/156.0" "-"

172.17.0.1 - - [21/Sep/2026:12:15:03 +0000] "GET / HTTP/1.1" 304 0 "-"
```

```
"Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:156.0) Gecko/20100101
Firefox/156.0" "-"
```

```
PS C:\Users\dimam> docker ps

CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS          PORTS                                    NAMES

49a40c9f612d   nginx:alpine   "/docker-entrypoint.…"   10 minutes ago   Up 17
seconds   0.0.0.0:8088->80/tcp, [::]:8088->80/tcp   lab-web-morgun
```

6. Запускаем контейнер short-task для выполнения echo команды. После ее выполнения, контейнер уходит в Exited(0), так как завершил главный процесс (показано в выводе команды docker ps -a, было в лекции 2, пункт 8)

```
PS C:\Users\dimam> docker run --name short-task alpine echo "I am your father"

Unable to find image 'alpine:latest' locally

latest: Pulling from library/alpine

797dd00a0fc7: Download complete

6d0606d1815c: Download complete

Digest:
sha256:294b683cb724975bec92580e1e685676bd4b50bda910ddb8c51d4cabeaec77e6

Status: Downloaded newer image for alpine:latest

I am your father
```

```
PS C:\Users\dimam> docker ps -a

CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS                        PORTS     NAMES

3795864ccd18   alpine         "echo 'I am your fat…"   5 seconds ago    Exited
(0) 4 seconds ago                 short-task

49a40c9f612d   nginx:alpine   "/docker-entrypoint.…"   12 minutes ago   Exited
(0) About a minute ago            lab-web-morgun
```

7. Docker inspect выводит полную информацию о контейнере в виде JSON файла (лекция 3, пункт 7). Из всего огромного вывода оставил только информацию о точках монтирования и портах

```
PS C:\Users\dimam> docker inspect lab-web-morgun
        "Mounts": [

            {

                "Type": "bind",

                "Source": "D:\\docker-lab1\\site",

                "Destination": "/usr/share/nginx/html",

                "Mode": "",

                "RW": true,

                "Propagation": "rprivate"

            }

        ],


       "PortBindings": {

                "80/tcp": [

                    {

                        "HostIp": "",

                        "HostPort": "8088"

                    }

                ]

            },



       "ExposedPorts": {
```

```
                "80/tcp": {}

            },


        "Ports": {},
```

8. Останавливаем контейнер через docker stop (лекция 2, пункт 3) и удаляем его с помощью docker rm (лекция 2, пункт 4). Затем удаляем локальный образ через docker rmi (лекция 2, пункт 6). Если бы контейнер, созданный из этого образа, не был предварительно остановлен и удалён, Docker выдал бы ошибку конфликта и не позволил бы удалить образ (лекция 2, пункт 6). В конце лабораторной останавливаем все контейнеры и удаляем их. Образ alpine также стираем (без ошибок, так как все созданные на его основе контейнеры удалены)

```
PS C:\Users\dimam> docker stop lab-web-morgun

lab-web-morgun

PS C:\Users\dimam> docker rm lab-web-morgun short-task

lab-web-morgun

short-task

PS C:\Users\dimam> docker rmi alpine

Untagged: alpine:latest

Deleted:
sha256:294b683cb724975bec92580e1e685676bd4b50bda910ddb8c51d4cabeaec77e6

PS C:\Users\dimam>
```

Мини квиз:

1. Что произойдёт с данными, созданными **внутри контейнера**, если удалить контейнер без тома?
   Все данные пропадут (лекция 3, пункт 5)
2. Чем отличается порт **хоста** от порта **в контейнере**?
   Порт внутри контейнера используется только внутри контейнера. Через флаг -p мы

связываем его с внешним портом, который можно использовать извне (для того же localhost)

3. Для чего нужна пара флагов интерактивного запуска, и когда одного из них достаточно?
   Флаг -i оставляет ввод открытым и позволяет передавать команды внутрь, а -t подключает псевдотерминал для корректного отображения командной строки и вывода (лекция 3, пункт 2). Одного флага -i достаточно, если псевдотерминал не нужен (можно передать команду через |). Один -t может использоваться, если нужен только вывод в терминале (например, логов)
4. Что показывает `Mounts` в `inspect` и как понять, что это именно bind-mount?
   Mounts показывает список всех смонтированных в контейнер хранилищ (директорий, файлов или томов), их пути и параметры подключения. bind-mout будет указан в Type как: "Type": "bind"
5. Почему образ может не удаляться, и что нужно сделать перед удалением?
   Образ может не удаляться, если есть контейнеры, основанные на нем. Для удаления образа нужно остановить и удалить все такие контейнеры (лекция 2, пункты 3,4,6)

P.S. двоичный демон из канарейки (условия лабораторной работы, между заданиями 1 и 2)